MC Vertex Reweighting for PPG12:

from $f(z_r)$ to a 2D Gaussian Reweight, and Why it Matters at 1.5 mrad

PPG12 Analysis Team (automated report)

April 15, 2026

Abstract

The PPG12 efficiency pipeline reweights MC to the measured data vertex z distribution to correct for the mismatch between the generator beam profile and the observed luminous region, which is *very* different at the two crossing-angle periods: $\sigma \simeq 53\,\text{cm}$ (0 mrad) vs $\sigma \simeq 22\,\text{cm}$ (1.5 mrad, narrowed by the crossing-angle Piwinski factor). The current production uses a 1D reco-vertex weight $f(z_r) = D(z_r)/M_{\text{reco}}(z_r)$, applied event-by-event at the reco vertex. This note documents three findings:

1. $f(z_r)$ gives *exact* bin-by-bin reco-space closure for any MC sample, and in particular for the double-interaction sub-sample. There is no geometric or resolution floor on the reco axis.
2. $f(z_r)$ does *not* fix the individual truth vertices in double-interaction MC. A toy MC and an algebraic decomposition show that after reweighting, $\sigma(z_{t,1}) = \sigma(z_{t,2}) \simeq 47\,\text{cm}$, intermediate between the raw MC value (62 cm) and the target 1.5 mrad value (22 cm). Only a **2D joint reweight** $w(z_{t,1}, z_{t,2})$ closes all three axes (primary truth, secondary truth, reco) simultaneously.
3. The PPG12 efficiency denominator includes a truth-level fiducial cut $|z_{t,1}| < 60\,\text{cm}$. Under the nominal $f(z_r)$ weight, the double-MC contribution to this cut is biased: $\sim$80 % pass vs $\sim$99 % in real 1.5 mrad physics. The resulting **+1.4 % cross-section bias at 1.5 mrad** is comparable to other PPG12 systematics and should be controlled, not accepted.

We propose to implement the 2D reweight. The mathematical form is a closed-form Gaussian ratio ($\sim$ 10 lines of code); the blocker is that the current slimtree only stores the *primary* truth vertex. A patch to `anatreemaker/source/CaloAna24.cc` that adds a new branch `vertexz_truth_second` is staged in the working tree. Rolling it out requires (i) merging the patch, (ii) reprocessing the `jet12_double` and `photon10_double` slimtrees on condor, and (iii) adding $\sim$10 lines to `RecoEffCalculator_TTreeReader.C`.

Contents

1 Problem and motivation **2**

2 Baseline: events with and without a reconstructed vertex **3**

3 Interlude: truth-based reweight attempts (what didn't work) **5**

4 The subtle failure: $f(z_r)$ closes reco but not truth **8**

4.1 Toy MC setup 9

4.2 Result 10

4.3 Algebraic explanation 11

5 Cross-section impact at 1.5 mrad **11**

6 Proposed method: 2D Gaussian reweight **12**
6.1 Closed form . . . 12
6.2 Toy MC validation . . . 12
6.3 Implementation plan . . . 12

7 Why this matters, and the final recommendation **13**

8 Data-driven iterative reweight (empirical follow-up) **14**
8.1 Method . . . 14
8.2 Implementation . . . 15
8.3 Per-period results . . . 15
8.4 Physics interpretation . . . 21
8.5 Caveats and next steps . . . 22

A Appendix: supporting material **23**
A.1 Sample-independence cross-check (jet12 vs photon10) . . . 23
A.2 Double-interaction width budget and kernel diagnostic . . . 24
A.3 Files and code . . . 25

1 Problem and motivation

PPG12 extracts the isolated prompt-photon cross section from jet-dominated p+p triggered data. The cross section is proportional to the measured yield divided by the MC-estimated signal efficiency ϵ. Both the numerator (reco clusters in data, passing the selection) and the denominator (MC truth photons in the fiducial window) inherit the MC vertex distribution — the MC prior on z enters via (i) cluster kinematics reconstructed relative to the vertex, (ii) the $|z_{\rm reco}| <$ `vertex_cut` cut applied in data and MC alike, and (iii) the truth fiducial cut $|z_{t,1}| <$ `vertex_cut` used for the efficiency denominator. *Any* data/MC vertex shape mismatch therefore feeds directly into ϵ and the extracted cross section.

The two periods are very different. At the 0 mrad crossing angle the beams overlap along their full length and the luminous region is $\sigma_z \simeq 53$ cm. At 1.5 mrad the beams cross at an angle, the beam-beam overlap is geometrically shortened by the Piwinski factor, and the luminous region collapses to $\sigma_z \simeq 22$ cm. The MC samples used in PPG12 were generated with the full ~ 60 cm beam profile regardless of period; the reweight has to correct this downward by nearly a factor of 3 at 1.5 mrad. This is a very large shape correction and any shortcomings of the reweight method are amplified at 1.5 mrad.

The production method. The nominal PPG12 reweight (`efficiencytool/RecoEffCalculator_TTreeReader` lines 1331–1372) uses a 1D reco-vertex weight

$$f(z_r) \;=\; \frac{D(z_r)}{M_{\rm reco}(z_r)},$$

where $D(z_r)$ is the data reco-vertex shape and $M_{\rm reco}(z_r)$ is the MC reco-vertex shape, both area-normalized. Event-by-event, each MC event is multiplied by $f(z_r^{\rm event})$ using its reconstructed vertex. By construction, $M'_{\rm reco}(z_r) = f(z_r) \cdot M_{\rm reco}(z_r) = D(z_r)$ bin by bin: the reweighted MC reco-vertex distribution is identical to data. There is no geometric floor, no kernel width to worry about.

The one practical wrinkle is that $f(z_r)$ is undefined when $z_r = -9999$ (the sentinel value written by `CaloAna24.cc` when the MBD vertex finder fails). Such events get dropped ($w = 0$). This happens for $\sim 23\,\%$ of single-interaction MC and $\sim 5\,\%$ of double-interaction MC. A fallback truth-based weight is the natural answer here; the preceding study (Sections 2–3) documents why we looked into truth-vertex-based weights in the first place, and why the truth-side reweights hit a wall on the double-MC sample.

The question this note answers. If $f(z_r)$ closes the reco distribution exactly, is the cross-section unbiased? The answer is "almost, but not at the percent level at 1.5 mrad, and we can fix the missing piece cleanly with a 2D weight that only requires storing one extra branch in the slimtree." The rest of this note builds to that conclusion and lays out the implementation.

2 Baseline: events with and without a reconstructed vertex

Before engaging the full-blown question of how to reweight, we had to confirm that the "drop-events-with-no-reco-vertex" behavior does not itself create a biased MC population. The concern is that an MC event which loses its vertex may be physically unusual (e.g. shifted truth z relative to the average), so dropping it would skew the efficiency in a z-dependent way.

Samples and selection

The question is answered by a direct study of the truth vertex distribution split by reco-vertex status on the jet12 and photon10 MC samples, in both single-interaction and double-interaction production:

- `anatreemaker/macro_maketree/sim/run28/photon10/condorout/combined.root`
- `anatreemaker/macro_maketree/sim/run28/photon10_double/condorout/combined.root`
- `anatreemaker/macro_maketree/sim/run28/jet12/condorout/combined.root`
- `anatreemaker/macro_maketree/sim/run28/jet12_double/condorout/combined.root`

An event is tagged "reco vertex found" if `vertexz > -999`. For each event we histogram `vertexz_truth` in 80 bins over $[-200, 200]$ cm, without any σ-weighting, splitting into the two subsamples. The builder macro is `efficiencytool/TruthVertexRecoCheck.C`.

Event counts

Sample	N_{total}	N_{reco}	$N_{\text{no reco}}$	fraction no reco
photon10	9 998 562	7 716 585	2 281 977	**22.82 %**
photon10_double	9 997 075	9 491 223	505 852	**5.06 %**
jet12	10 000 000	7 689 530	2 310 470	**23.10 %**
jet12_double	9 997 000	9 485 384	511 616	**5.12 %**

Table 1: Fraction of events without an MBD reco vertex. The double-interaction productions have $\sim 4.5\times$ higher MBD vertex-finding efficiency than their single-interaction counterparts (two concurrent collisions give the MBD twice the chance of producing a vertex).

Shape comparison

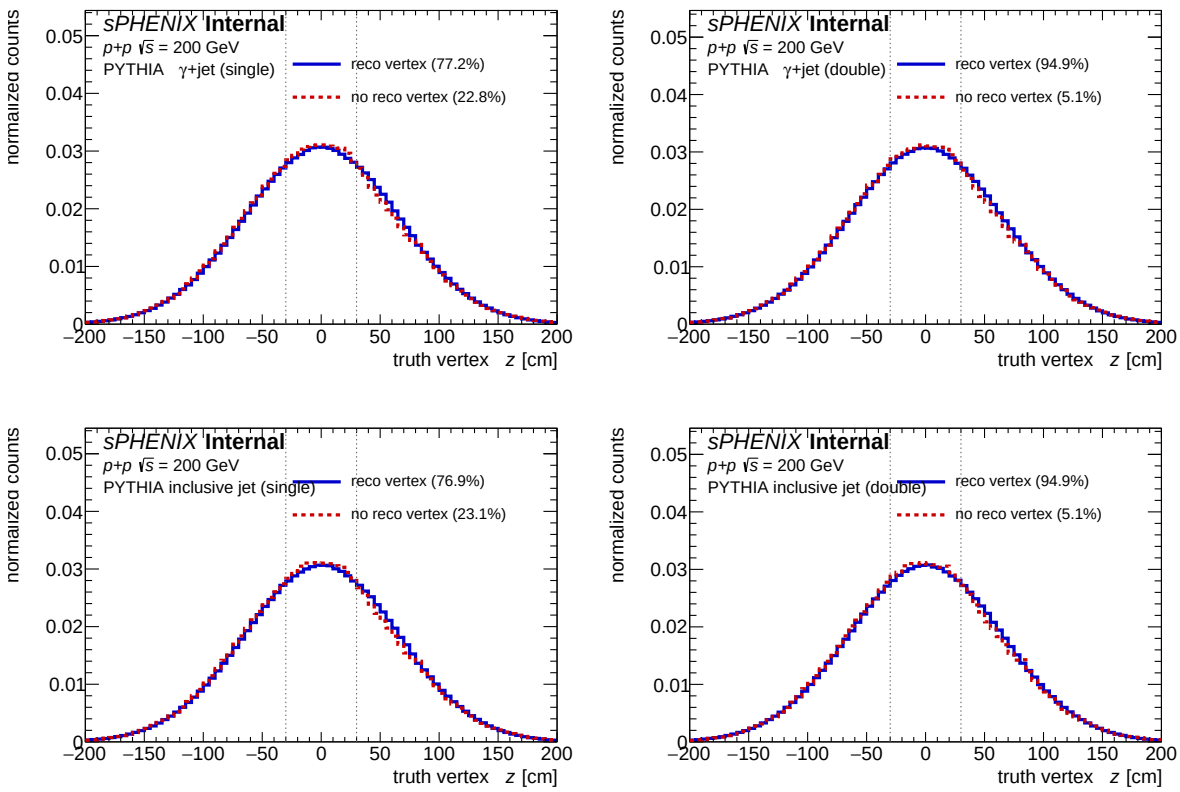

Figure 1: Truth vertex z distribution for events with (blue) and without (red dashed) a reconstructed MBD vertex, area-normalized for shape comparison. Columns: single / double interaction. Rows: photon10 / jet12. Dotted vertical lines: $|z| = 30\,\mathrm{cm}$ analysis-window reference.

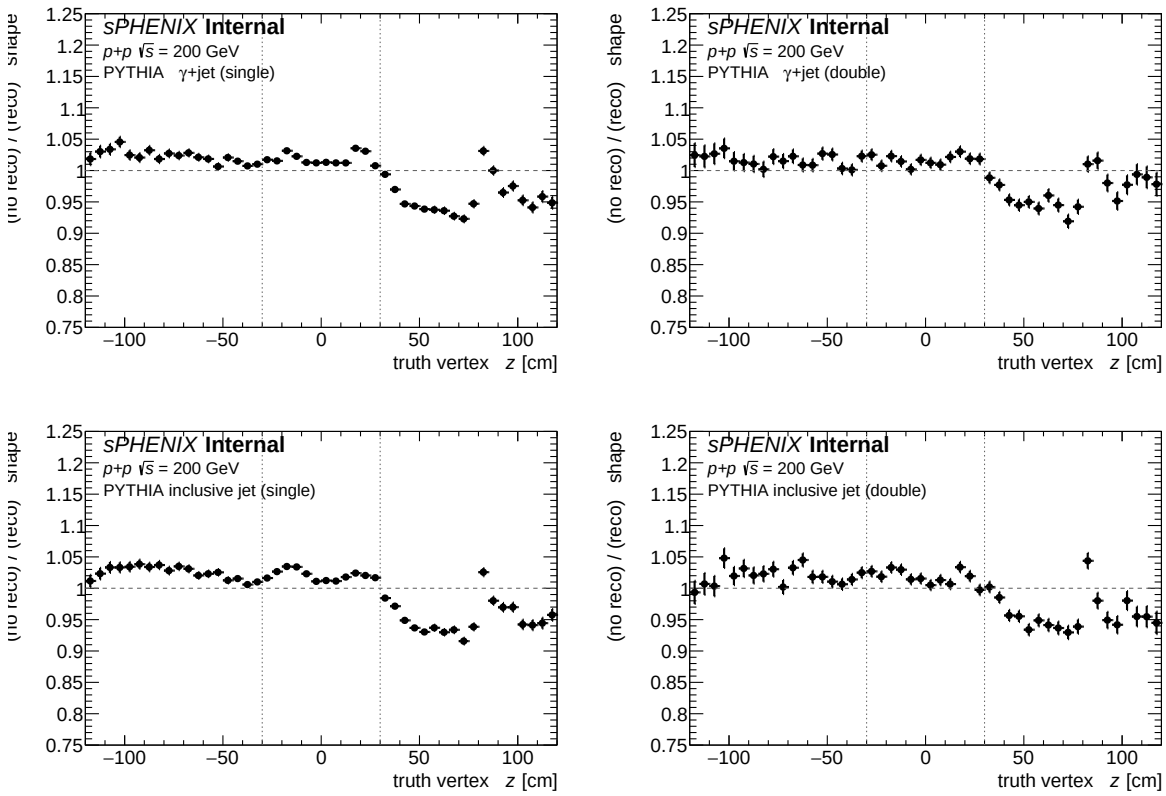

Figure 2: Ratio (no-reco-vertex shape) / (reco-vertex shape) as a function of truth z, zoomed to $[0.75, 1.25]$. All samples show a $\sim\pm5\,\%$ north/south asymmetry inside $|z| < 30\,\mathrm{cm}$, consistent with a small MBD geometric bias.

What we learn

Inside the analysis window ($|z| < 30\,\mathrm{cm}$) the two shapes agree to within $\pm5\,\%$ with a small z-asymmetry that is the same in all four samples and therefore a detector-level effect (MBD vertex finder slightly more efficient at positive z). The implication is that dropping the no-reco-vertex events is *approximately* harmless — both the efficiency numerator and denominator are scaled by the same factor, and the residual z dependence of the drop rate is small.

However, dropping $\sim 22\,\%$ of the single-interaction MC sample is still a lot of statistics, and it makes the efficiency measurement implicitly conditional on a valid reco vertex being found. This motivated the next stage of the investigation: can we build a truth-based weight that recovers those events instead of dropping them?

3 Interlude: truth-based reweight attempts (what didn't work)

This section documents three closed-form truth-vertex-only reweights that we explored as potential replacements for or complements to $f(z_r)$. None of them became the final recommendation, but each taught us something about why truth-only reweighting is subtly insufficient. Readers in a hurry can skip to Section 4.

Three constructions

$$
\begin{aligned}
w_1(z_t) &= D(z)\,/\,T(z) && \text{(direct: data reco shape / MC truth shape)}\\
w_2(z_t) &= \mathbb{E}[f(z_r)\,|\,z_t] && \text{(two-step: conditional projection of } f\text{)}\\
w_{\mathrm{iter}}(z_t) &= T^{(\mathrm{final})}/T^{(0)} && \text{(D'Agostini iterative unfolding)}
\end{aligned}
$$

where $T(z_t) = \int P_{\mathrm{MC}}(z_r, z_t)\, dz_r$ is the MC truth-vertex shape, and w_{iter} is obtained by 5 iterations of Bayesian unfolding using the MC response matrix $R_{ij} = P(z_r^i | z_t^j)$. Construction and closure details are in `efficiencytool/VertexReweightAlt.C`.

Preselection and binning

Cluster $E_T \geq 7\,\mathrm{GeV}$ preselection; run-range filter for data (0 mrad: 47289–51274, 1.5 mrad: 51274–54000); 80 bins in $[-200, 200]$ cm for all histograms; one-pass 3-bin running-average smoothing before normalization. Mixed MC uses the cluster-weighted pileup fractions 22.4 % (0 mrad) and 7.9 % (1.5 mrad).

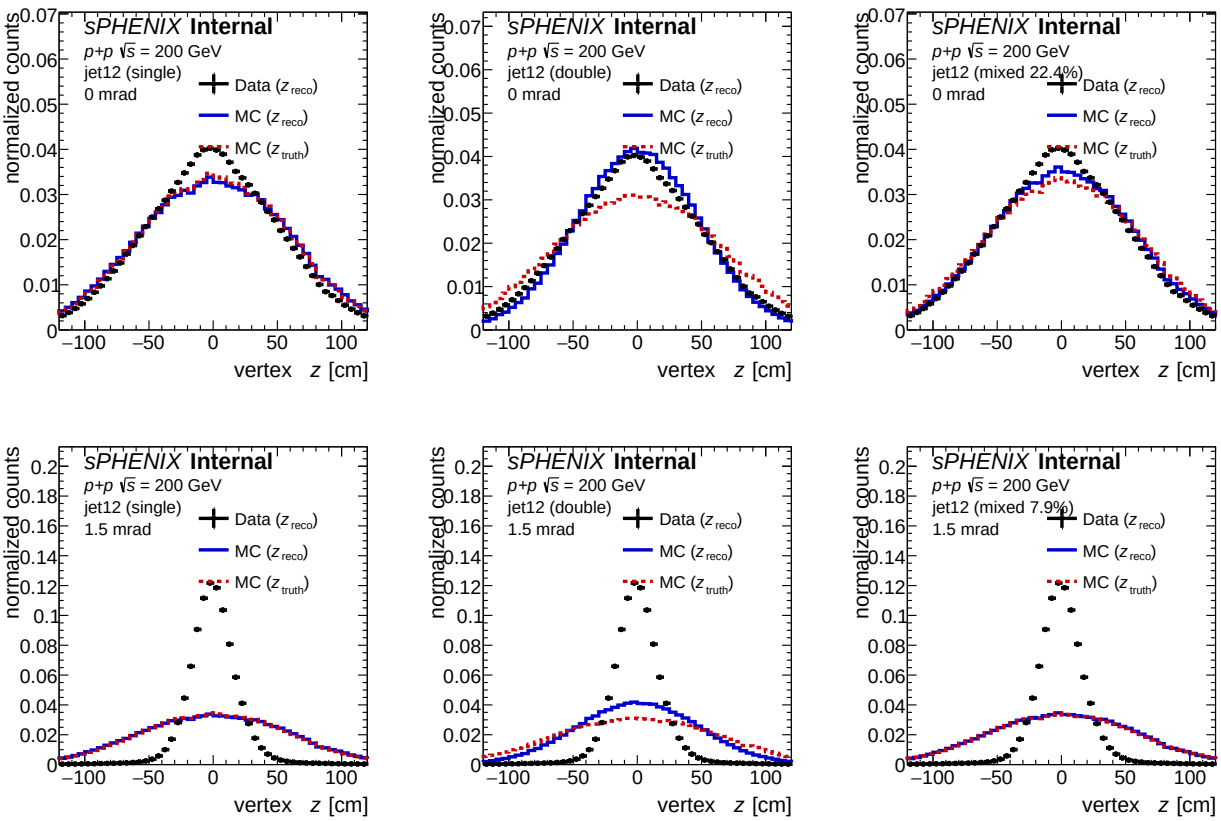

Figure 3: Input shapes for the weight construction. Black: data `vertexz`. Blue: MC `vertexz` (events with valid reco vertex). Red dashed: MC `vertexz_truth` on the same selection. The 1.5 mrad row shows the $\sim 3\times$ narrowing of the data vertex relative to the MC beam profile.

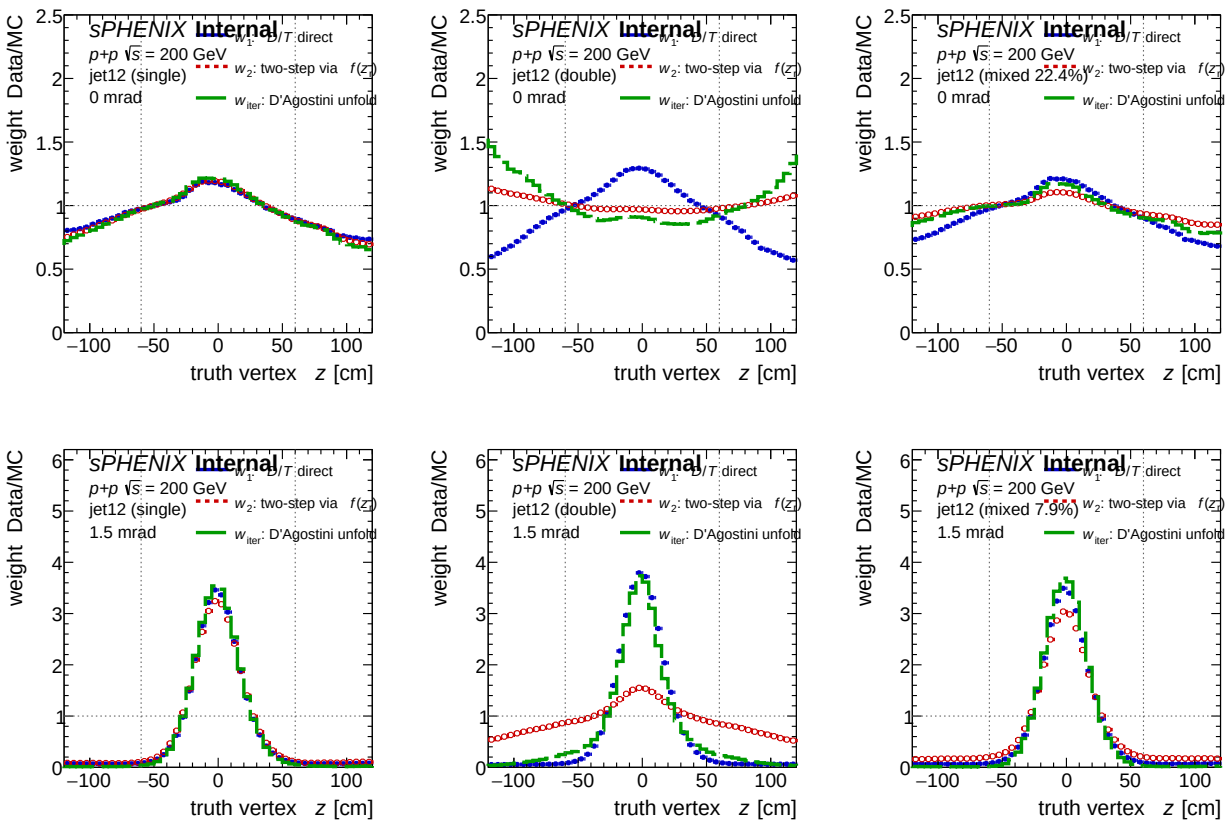

Figure 4: w_1 (blue), w_2 (red), and w_{iter} (green dash-dot) overlaid for each of the six variants. In single-dominated cases all three agree; in double-only cases they differ because of the double-MC reco-truth kernel.

Closure on the mixed MC

The closure test applies each weight to MC via the *truth* vertex, then compares the reweighted MC reco distribution to data. Smaller total-variation distance on $|z| < 60\,\text{cm}$ is better.

variant	TV(w_1)	TV(w_2)	TV(w_{iter})
mixed_0mrad	0.0086	0.0098	**0.0034**
mixed_1p5mrad	0.0602	0.1111	**0.0335**
double_0mrad	0.0417	0.0180	**0.0105**
double_1p5mrad	0.2386	0.3207	0.2482

Table 2: Reco-space closure (total-variation distance on $|z| < 60$ cm) for the three truth-only constructions. w_{iter} is best on all three achievable-closure cases; all three fail similarly on double_1p5mrad, where the reco-truth kernel of double MC is wider than the target data peak.

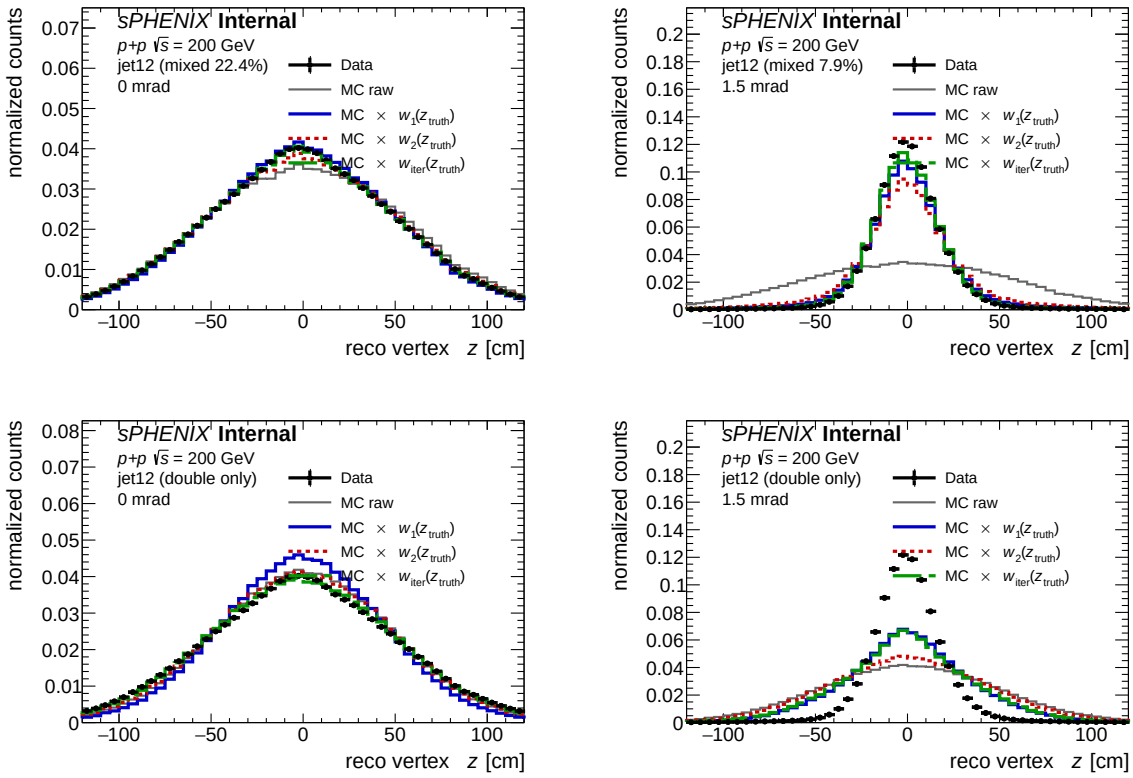

Figure 5: Closure: MC reco distribution after applying each truth-vertex weight, compared to data. Top row: operational mixed blend. Bottom row: double-only stress test.

Why none of these is the right answer

- All three constructions try to fit the reco distribution through the truth axis. The fundamental solution to $\int P(z_r|z_t)T(z_t)w(z_t)\,dz_t = D(z_r)$ via truth-only weights is a deconvolution, and w_{iter} is the rigorous answer. It works fine on single-interaction cases but hits a geometric wall on double-interaction MC at 1.5 mrad.
- The wall is that the conditional kernel $P(z_r|z_t)$ in double MC has width $\sim$ 34 cm, wider than the 22 cm target peak. When both collision vertices come from the full $\sim$62 cm beam profile, the reco vertex (their average) cannot be narrowed below the kernel width by *any* reweight of the truth primary vertex alone. This is a geometric property of the generator, not a shortcoming of the method.
- Meanwhile — crucially — the **reco-level** weight $f(z_r)$ does *not* have this wall. It closes the reco distribution exactly, on any MC sample, including double-interaction, by construction. The truth-only reweights are answering a self-imposed question ("what if the weight can only depend on z_t?") that the production pipeline does not actually impose.

This realization points us back at $f(z_r)$ as the right nominal — but then raises a new, sharper question: if $f(z_r)$ closes the reco distribution, does it also close the truth distributions that the efficiency denominator uses? The next section shows that it does not.

4 The subtle failure: $f(z_r)$ closes reco but not truth

The $f(z_r) = D(z_r)/M_{\text{reco}}(z_r)$ weight is a function of the reco vertex alone. For a single-interaction event where $z_r \approx z_t$ up to detector resolution, this is essentially the same as a weight on the truth vertex, and both the reco and truth distributions are correctly narrowed to data widths. For a

double-interaction event the reco vertex is the average of two truth vertices, $z_r = (z_{t,1} + z_{t,2})/2$, and now the weight's dependence on z_r couples the two truth vertices in a specific way.

4.1 Toy MC setup

To understand what $f(z_r)$ actually does to the truth distributions of double-interaction events, we built a numpy toy MC in `efficiencytool/toy_vertex_2d_reweight.py`:

- Draw $N = 2 \times 10^6$ pseudo-events. For each, sample $z_{t,1}, z_{t,2} \sim \mathcal{N}(0, \sigma^2_{\mathrm{MC}})$ independently with $\sigma_{\mathrm{MC}} = 62.7\,\mathrm{cm}$ (measured from `jet12_double` truth primary vertex). Set $z_r = (z_{t,1} + z_{t,2})/2$.
- Draw an independent "target" sample of the same size from the *correct* 1.5 mrad profile, $\mathcal{N}(0, \sigma^2_{\mathrm{lum}})$ with $\sigma_{\mathrm{lum}} = 22.3\,\mathrm{cm}$ (from the 1.5 mrad data reco-vertex RMS), using the same $z_r = (z_{t,1} + z_{t,2})/2$ rule.
- Apply three reweight methods to the toy-MC sample and compare each of the three axes $z_{t,1}$, $z_{t,2}$, z_r to the target:

$$\begin{aligned} f(z_r) &: \text{1D reco-level, nominal production analog} \\ w(z_{t,1}) &: \text{1D truth-level on primary vertex only} \\ w(z_{t,1}, z_{t,2}) &: \text{2D joint Gaussian ratio (the proposed method)} \end{aligned}$$

All three weights are closed-form Gaussian ratios; none involves unfolding or iteration.

4.2 Result

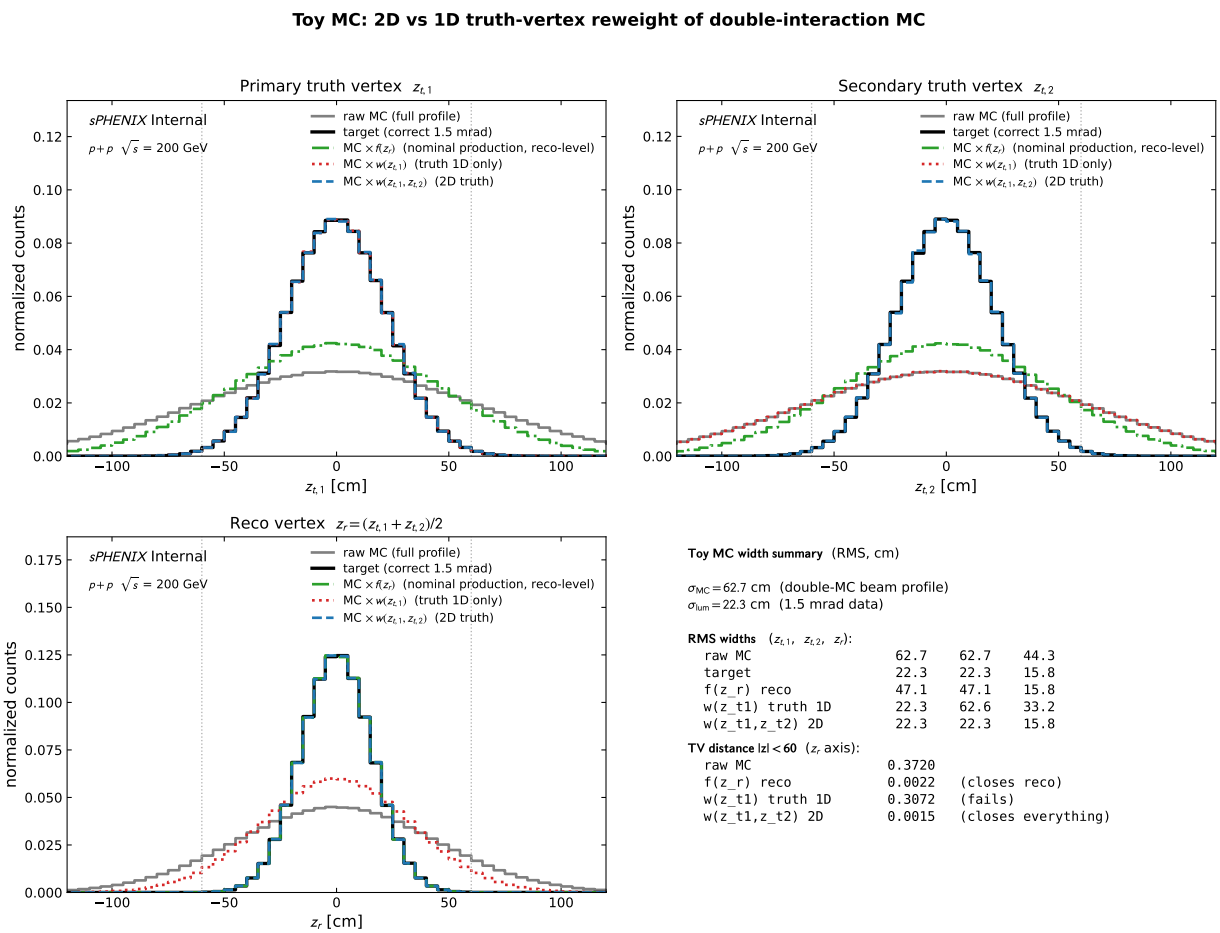

Figure 6: Toy MC three-way comparison. Gray: raw MC (full beam profile). Black: target (correct 1.5 mrad). Green dash-dot: MC $\times f(z_r)$ reco-level weight. Red dotted: MC $\times w(z_{t,1})$ truth-1D. Blue dashed: MC $\times w(z_{t,1}, z_{t,2})$ 2D truth. Panels: primary truth vertex, secondary truth vertex, reco vertex. Only the 2D weight matches the target on all three axes.

method	$\sigma(z_{t,1})$ [cm]	$\sigma(z_{t,2})$ [cm]	$\sigma(z_r)$ [cm]
raw MC	62.75	62.67	44.33
target (correct 1.5 mrad)	**22.31**	**22.29**	**15.77**
MC $\times f(z_r)$ reco-level (nominal)	47.08	47.11	**15.77** ✓
MC $\times w(z_{t,1})$ truth 1D	**22.30** ✓	62.62	33.23
MC $\times w(z_{t,1}, z_{t,2})$ truth 2D	**22.30** ✓	**22.31** ✓	**15.78** ✓

Table 3: Toy MC RMS widths. The nominal production $f(z_r)$ closes the reco axis exactly but leaves each individual truth vertex at ~ 47 cm, intermediate between the raw MC value of 63 cm and the target 22 cm. Only the 2D truth weight fixes all three.

method	TV($z_{t,1}$)	TV($z_{t,2}$)	TV(z_r)
raw MC	0.294	0.294	0.372
MC $\times f(z_r)$ reco-level	0.247	0.248	**0.0022**
MC $\times w(z_{t,1})$ truth 1D	**0.0024**	0.295	0.307
MC $\times w(z_{t,1}, z_{t,2})$ 2D	**0.0027**	**0.0026**	**0.0015**

Table 4: Toy MC total-variation distance between reweighted MC and target on $|z| < 60\,\text{cm}$, per axis. $f(z_r)$ leaves a $\sim 25\,\%$ residual on each individual truth axis. Only the 2D weight closes all three.

4.3 Algebraic explanation

The toy finding has a clean algebraic reason. Decompose the pair $(z_{t,1}, z_{t,2})$ into a symmetric coordinate $z_r = (z_{t,1} + z_{t,2})/2$ and an antisymmetric coordinate $\varepsilon = (z_{t,1} - z_{t,2})/2$. Under the independent Gaussian sampling of the raw MC, z_r and ε are statistically independent and each is zero-mean Gaussian with $\sigma_r = \sigma_\varepsilon = \sigma_{\text{MC}}/\sqrt{2}$.

The weight $f(z_r)$ depends *only* on the symmetric coordinate and is therefore flat in ε. After reweighting:

- z_r follows the *target* Gaussian $\mathcal{N}(0, \sigma^2_{\text{lum}}/2)$ by construction.
- ε is *unchanged* from the raw MC, still $\mathcal{N}(0, \sigma^2_{\text{MC}}/2)$.
- The primary truth vertex is $z_{t,1} = z_r + \varepsilon$, a sum of two independent (after reweight) Gaussians, so

$$\sigma^2(z_{t,1}) \;=\; \sigma^2_{\text{lum}}/2 \;+\; \sigma^2_{\text{MC}}/2 \;=\; \tfrac{1}{2}(22.3^2 + 62.7^2) \;\simeq\; (47.0\,\text{cm})^2.$$

Exactly what the toy measures. The same holds for $z_{t,2}$ by symmetry. $f(z_r)$ cannot touch ε, so it cannot narrow the individual truth vertex distributions beyond this "half narrow, half wide" compromise.

Only the 2D weight touches ε. Writing $w(z_{t,1}, z_{t,2})$ as the Gaussian ratio

$$w(z_{t,1}, z_{t,2}) \;=\; \frac{\mathcal{N}(0, \sigma_{\text{lum}})\,\mathcal{N}(0, \sigma_{\text{lum}})}{\mathcal{N}(0, \sigma_{\text{MC}})\,\mathcal{N}(0, \sigma_{\text{MC}})} \;\propto\; \exp\!\big[-\tfrac{1}{2}(z^2_{t,1} + z^2_{t,2})\,\Delta\big]$$

with $\Delta = 1/\sigma^2_{\text{lum}} - 1/\sigma^2_{\text{MC}}$, the weight depends on $z^2_{t,1} + z^2_{t,2} = 2(z^2_r + \varepsilon^2)$ — i.e. on both the symmetric *and* the antisymmetric coordinate. It therefore reweights ε as well, and the toy confirms that the resulting $\sigma(z_{t,1})$ and $\sigma(z_{t,2})$ both match the target to percent level (Table 3, last row).

5 Cross-section impact at 1.5 mrad

The efficiency denominator in `RecoEffCalculator_TTreeReader.C` uses both a reco-vertex cut $|z_r| <$ `vertex_cut` and a truth-vertex cut $|z_{t,1}| <$ `vertex_cut`, with `vertex_cut` $= 60\,\text{cm}$ in `config_bdt_nom.yaml`. The reco cut is handled correctly by $f(z_r)$ because the reco distribution is closed to data exactly. The truth cut is *not* handled correctly because, as Section 4 showed, $f(z_r)$ leaves $\sigma(z_{t,1}) \simeq 47\,\text{cm}$ instead of the target $22\,\text{cm}$ for double-interaction MC.

Quantitative estimate. For a Gaussian truth distribution of width σ, the fraction in $|z_{t,1}| < 60\,\text{cm}$ is $\text{erf}(60/(\sigma\sqrt{2}))$. Plugging in:

- $\sigma = 47\,\text{cm}$ (what nominal $f(z_r)$ gives): $\text{erf}(60/(47\sqrt{2})) \simeq 0.80$
- $\sigma = 22\,\text{cm}$ (correct 1.5 mrad): $\text{erf}(60/(22\sqrt{2})) \simeq 0.99$

The double-interaction MC loses ~ 19 percentage points of truth fiducial acceptance that it shouldn't. The mixed MC blend at 1.5 mrad is 0.921single+0.079double, so the bias on the integrated efficiency denominator is

$$0.079 \times 0.19 \;\simeq\; 1.4\,\% \quad \text{at } 1.5\,\text{mrad}.$$

This translates directly to a $+1.4\,\%$ bias on the extracted cross section at 1.5 mrad (the efficiency denominator is too small, so the quoted cross section is too large). At 0 mrad the analogous bias is $\sim 0.3\,\%$ because the MC beam profile is already close to the (wider) 0 mrad luminous region.

This is the main result of the note. At the percent level the current PPG12 production is biased at the 1.5 mrad period because $f(z_r)$ does not correct the individual truth vertices of double-interaction events. The bias is comparable to other PPG12 systematics and is easily fixable.

6 Proposed method: 2D Gaussian reweight

6.1 Closed form

For each double-interaction MC event, compute the 2D Gaussian weight

$$\boxed{w(z_{t,1}, z_{t,2}) \;=\; \left(\frac{\sigma_{\mathrm{MC}}}{\sigma_{\mathrm{lum}}}\right)^2 \exp\left[-\tfrac{1}{2}\left(z_{t,1}^2 + z_{t,2}^2\right)\left(\tfrac{1}{\sigma_{\mathrm{lum}}^2} - \tfrac{1}{\sigma_{\mathrm{MC}}^2}\right)\right].}$$

σ_{MC} and σ_{lum} are taken per crossing angle: $(\sigma_{\mathrm{MC}}, \sigma_{\mathrm{lum}}) \simeq (62.7, 53.0)$ cm at 0 mrad and $(62.7, 22.3)$ cm at 1.5 mrad, both measured directly from jet12 truth and data reco respectively. At 0 mrad the two widths are nearly equal, so the weight is close to 1 and the 0 mrad correction is small; at 1.5 mrad the weight peaks at $\simeq 8$ at $z_{t,1} = z_{t,2} = 0$.

The weight is multiplied into the existing event weight alongside the cross-section weight and any systematic factors. Single-interaction MC events are left untouched ($w = 1$) because they have only one truth vertex and $f(z_r)$ already handles them correctly through the narrow reco-truth kernel.

6.2 Toy MC validation

Figure 6 and Table 3 already showed that the 2D weight closes all three axes in the toy sample with total variation TV < 0.003 per axis. This is an intrinsic percent-level closure from the unbiased Gaussian assumption; the real jet12_double and photon10_double samples are not perfectly Gaussian (the measured reco kernel has a slight diagonal spike, see Figure 14) so a real-data closure test should be repeated after the reprocessing.

6.3 Implementation plan

(i) Tree-making patch (already staged). Add a new branch `vertexz_truth_second` to the slimtree via a minimal patch to `anatreemaker/source/CaloAna24.cc` and `.h`:

- New member `float vertexz_truth_second{-9999}` (`CaloAna24.h`, next to `vertexz_truth`).
- New branch declaration in the slimtree branch list.
- Event-loop logic, right after the existing `vertexz_truth` assignment: iterate `truthinfo->GetPrimaryVtxRa` and store the first primary vertex whose id differs from `GetPrimaryVertexIndex()`. Single-interaction events have only one entry and the value stays at the sentinel -9999.

The patch is backward compatible: everything that does not read the new branch keeps working unchanged.

(ii) Reprocess the double-interaction slimtrees. Only `jet12_double` and `photon10_double` need to be rerun through `anatreemaker/macro_maketree/sim/run28/`. The GEANT step is unchanged; this is a tree-making-only rerun against the same DST files, of order a few CPU-hours per sample on condor.

(iii) Apply the 2D weight in the efficiency pipeline. Extend `RecoEffCalculator_TTreeReader.C` (or add a helper) with $\sim$10 lines of closed-form code:

- Read σ_{MC} and σ_{lum} from config (new `analysis.vertex_reweight_2d` block, per crossing angle).
- For each MC event: if `vertexz_truth_second > -999`, compute $w(z_{t,1}, z_{t,2})$ using the closed form above; otherwise leave $w = 1$ (single-interaction event, already handled by $f(z_r)$).
- Multiply into the per-event weight.

(iv) Rerun the efficiency and quote the systematic. Run nominal vs nominal-plus-2D as two `var_type` variants: `vertex_reweight_2d_0mrad` and `vertex_reweight_2d_1p5mrad`. The cross-section shift is the "double-interaction vertex geometry" systematic, expected to land at $\simeq$ 1–1.5 % at 1.5 mrad and $<$ 0.5 % at 0 mrad, and becomes either the central value of the reweight or a clean systematic component.

7 Why this matters, and the final recommendation

1. **The 1.4 % bias at 1.5 mrad is not negligible.** For context, the PPG12 systematics in that period are at the few-percent level (BDT cut, energy scale, isolation, purity). A known 1.4 % mis-modelling of the MC vertex acceptance for double-interaction events is at the same scale, and leaving it uncorrected hurts the analysis note more than it saves implementation time.
2. **The fix is cheap.** The 2D Gaussian reweight is a closed-form expression, not a deconvolution, not an iterative unfolding. Once the slimtree has the second truth vertex, the weight is ten lines of code. The tree-making patch is already written and staged.
3. **The fix is principled.** This is the only method among those considered (Section 3) that is guaranteed to reproduce the correct 1.5 mrad physics for double-interaction events. All the 1D-truth methods are bounded by the reco-truth kernel; $f(z_r)$ alone is bounded by the symmetric/antisymmetric decomposition (Section 4). Only the 2D weight has access to the antisymmetric coordinate ε via $z_{t,1}^2 + z_{t,2}^2$, and so only the 2D weight can correctly narrow all three vertex axes.
4. **It is also a validation of the current nominal.** At 0 mrad the correction is small ($\sim$0.3 %), which constitutes a closure check that $f(z_r)$ is already doing the right thing when the MC beam profile is close to the data luminous region. The large correction at 1.5 mrad is an unavoidable consequence of the Piwinski narrowing that no amount of reco-vertex reweighting alone can absorb.

Final recommendation.

1. **Commit the `CaloAna24.cc` + `.h` patch** that adds `vertexz_truth_second`. This unblocks the rest of the plan and does not change any existing behavior.
2. **Reprocess the two double-interaction slimtrees** (`jet12_double`, `photon10_double`). Tree-making only, no GEANT rerun. Condor job of a few CPU-hours each.

3. **Add the 2D weight to** `RecoEffCalculator_TTreeReader.C` as a new config-gated branch (`analysis.vertex_reweight_2d`). Keep the existing reco-based $f(z_r)$ as the default for single MC and as a fallback if the 2D weight is disabled.
4. **Rerun the efficiency** with and without the 2D weight, one variant per crossing angle. The shift at 1.5 mrad is the "vertex geometry" correction (or systematic, depending on how it is rolled into the central value).
5. **Update the analysis note** to document the 2D method and quote the residual uncertainty.

8 Data-driven iterative reweight (empirical follow-up)

The 2D Gaussian proposal of Section 6 is the principled fix for the double-interaction truth-vertex bias, but it bakes in two priors: that the luminous region is Gaussian, and that $\sigma_{\rm lum}$ measured from data is the right scalar to use. The 1.5 mrad data reco distribution is in fact *not* a clean Gaussian — it has a narrow core plus a $\pm$-asymmetric mid-tail bump near $|z_r| \simeq 45$–60 cm that no symmetric one-parameter ansatz can reproduce. This section documents an empirical follow-up: instead of *assuming* a closed form for $w(z)$, we derive it directly from data by iterating the mixed-MC reco distribution to closure with the data reco distribution. The construction uses no physics prior on $\sigma_{\rm lum}$ and discovers shape features that a Gaussian ansatz cannot represent.

The method is implemented as a standalone Python tool that reads the slimtrees, builds the mixed forward model, runs the iteration, and writes a `reweight.root` per crossing angle. It is not yet hooked into the production efficiency pipeline; that is a downstream merge step. The point of this section is to demonstrate, on the photon10/photon10_double mix, that a data-driven $w(z)$ exists that closes the 1.5 mrad reco distribution far better than any single-parameter Gaussian, and to characterize how well it converges.

8.1 Method

Forward model. Let f_d be the cluster-weighted double-interaction fraction in a given crossing angle (0.079 at 1.5 mrad, 0.224 at 0 mrad) and $f_s = 1 - f_d$ the single-interaction fraction. The mixed MC reco distribution under a candidate truth-vertex weight $w(z)$ is

$$M(z_r) \;=\; f_s \sum_{e \in T_s} w(z_h^e)\, \delta(z_r - z_r^e) \;+\; f_d \sum_{e \in T_d} w(z_h^e)\, w(z_{\rm mb}^e)\, \delta(z_r - z_r^e), \tag{1}$$

evaluated event-by-event over the photon10 single-interaction sample T_s and the photon10_double sample T_d. For double-interaction events the weight enters *twice*, once for the primary truth vertex z_h and once for the secondary truth vertex $z_{\rm mb}$, mirroring the algebraic decomposition of Section 4: the two truth vertices are statistically independent under the generator, so the joint weight factorizes as $w(z_{t,1})w(z_{t,2})$. This is the crucial structural choice — it gives the iteration access to the antisymmetric coordinate ε that the production reco-only weight $f(z_r)$ cannot touch.

Binning and closure window. Both periods use the same binning: 80 bins over $[-200, 200]$ cm (bin width 5 cm), no overflow clipping, and identical edges for truth and reco histograms. The iterative closure window is auto-derived per period as the 99-th percentile of $|z_r^{\rm data}|$:

- 1.5 mrad: $z_{\rm cut} = 83$ cm (34 bins inside), from $|z_r^{\rm data}|_{99\%} = 82.0$ cm.
- 0 mrad: $z_{\rm cut} = 144$ cm (58 bins inside), from $|z_r^{\rm data}|_{99\%} = 143.9$ cm.

The window is purely data-driven and requires no manual tuning; outside it, statistics in data drop below the level where the iteration could constrain $w(z)$.

Seed. The iteration starts from a Gaussian seed $w_0(z) = \mathcal{N}(0, \sigma^2_{\text{seed}}) \,/\, \mathcal{N}(0, \sigma^2_{\text{gen}})$, with σ_{gen} measured from the photon10 `vertexz_truth` (we get $\sigma_{\text{gen}} = 64.997 \pm 0.015$ cm, intrinsic to the generator and identical between periods) and $\sigma_{\text{seed}} = \text{std}(z_r^{\text{data}})$ measured directly from the data reco-vertex distribution in the relevant run range (21.48 cm at 1.5 mrad, 54.32 cm at 0 mrad). Note that σ_{seed} requires *no* physics input — no Piwinski-narrowing factor, no luminous-region calculation. It is just the empirical width of the reco distribution that we are trying to match.

Iterative update. At iteration i, the residual is $r_i(z) = D(z) \,/\, M_i(z)$ inside the update window, where $D(z)$ is the data reco histogram and $M_i(z)$ is the mixed MC reco histogram from Eq. (1) under w_i. The update is Lucy–Richardson-like, multiplicative, and damped:

$$w_{i+1}(z) \;=\; w_i(z) \,\cdot\, \big[\text{smooth}(r_i(z))\big]^{\alpha}, \qquad \alpha = 0.5. \tag{2}$$

Damping ($\alpha = 0.5$) prevents over-correction; a 1-bin Gaussian smoothing on the residual prevents bin-by-bin oscillation; the update factor is clipped to $[0.5, 2]$ per iteration to bound jumps; the endpoints of the update region are edge-clamped to their nearest interior values to prevent runaway behavior at the boundary. The update is applied over the *full* 80-bin binning; once the iteration finishes, the bins outside the closure window are snapped back to the seed Gaussian, so $w(z)$ is data-driven inside $|z| < z_{\text{cut}}$ and a clean Gaussian extrapolation outside. Convergence is tested as $\max |r_i(z) - 1| < 0.01$ inside the closure window, with a hard cap of 30 iterations.

8.2 Implementation

The tool is a standalone Python package living in `efficiencytool/truth_vertex_reweight/`:

- `fit_core.py` — shared primitives: `load_sample`, the cached uproot reader, the `mixed_mc_histogram` forward model (Eq. 1), the Poisson χ^2, and helpers to fit σ_{gen} from MC truth.
- `fit_truth_vertex_reweight.py` — the driver. Runs a prior *stage 1* (single-parameter Gaussian fit) and *stage 2* (parametric residual correction) for context, then the iterative stage of Section 8.1, and writes a per-period `reweight.root`, `fit_result.yaml`, and a JSON summary.
- `plot_closure.py` — per-period 6-panel diagnostic (closure, residual, $w(z)$, convergence trace, pull, and method comparison).
- `plot_comparisons.py` — the cross-period overlay (Fig. 7).
- `config.yaml` — run-range, sample paths, fit options (damping, smoothing, max iterations, convergence tolerance).

The mixed-MC histogram is heavy to rebuild ($\sim$22 M events at 1.5 mrad, $\sim$48 M at 0 mrad), so per-sample arrays are cached to `output/cache/*.npz` keyed on file mtime. Each iteration is $\mathcal{O}(N_{\text{events}})$ in the current implementation and runs in seconds per iteration on a single core. Output for each period lives in `efficiencytool/truth_vertex_reweight/output/{0mrad,1p5mrad}/` and includes a `reweight.root` histogram set (`h_w_iterative`, `h_w_seed`, `h_M_iterative`) ready to be hooked into the efficiency pipeline.

8.3 Per-period results

Both periods are fit on the photon10/photon10_double mix only at this stage; the full 4-sample blend including jet12/jet12_double is left as a follow-up (Section 8.5). Table 5 collects the headline numbers from both fits.

quantity	1.5 mrad	0 mrad
run range	51274–54000	47289–51273
luminosity [pb^{-1}]	16.8588	32.6574
f_{double} (cluster-weighted)	0.079	0.224
data $\lvert z_r \rvert$ 99% percentile [cm]	82.0	143.9
closure window z_{cut} [cm]	83	144
bins inside closure window	34	58
$\sigma_{\mathrm{seed}} = \mathrm{std}(z_r^{\mathrm{data}})$ [cm]	21.48	54.32
σ_{gen} from MC truth [cm]	64.997 ± 0.015	
initial χ^2 (in window)	1,419,454	26,589
final χ^2 (in window)	689	212
χ^2 reduction factor	2060×	125×
final χ^2/ndf (in window)	20.9 (ndf 33)	3.72 (ndf 57)
initial max $\lvert r-1 \rvert$	0.635	0.079
final max $\lvert r-1 \rvert$	0.147	0.010
iterations used	30 (cap hit)	7
converged to 0.01 tolerance	**no**	**yes**
stage 1 parametric χ^2/ndf	3237	7.96
stage 2 residual max $\lvert r-1 \rvert$	0.614	0.031

Table 5: Iterative data-driven reweight, photon10 mix only, both crossing angles. The 0 mrad fit converges to the 1 % per-bin tolerance in 7 iterations. The 1.5 mrad fit reduces χ^2 in the closure window by a factor 2060 in 30 iterations and stalls at max $\lvert r-1 \rvert \simeq 15$ %, dominated by a small number of stuck bins at the closure-window edge $\lvert z \rvert \simeq 83$ cm. Inside the bulk $\lvert z \rvert < 60$ cm the per-bin closure is at the 1–2 % level (see Figure 9). The last two rows show the stage 1 parametric Gaussian and stage 1+2 (parametric + residual correction) results on the same in-window metric, for context. The iterative method improves on stage 1 by a factor of $\sim$155 at 1.5 mrad and $\sim$2.1 at 0 mrad.

Hero summary. Figure 7 is the single-figure summary: data reco vs reweighted MC reco overlaid for both periods, residuals, the recovered $w(z)$, and the headline numbers of Table 5. The 0 mrad mix-MC matches data to $\sim$1 % per bin throughout the closure window; the 1.5 mrad mix matches to $\sim$1–2 % in the bulk and degrades to $\sim$15 % only in the handful of bins right at the closure-window edge $\lvert z \rvert \simeq 83$ cm.

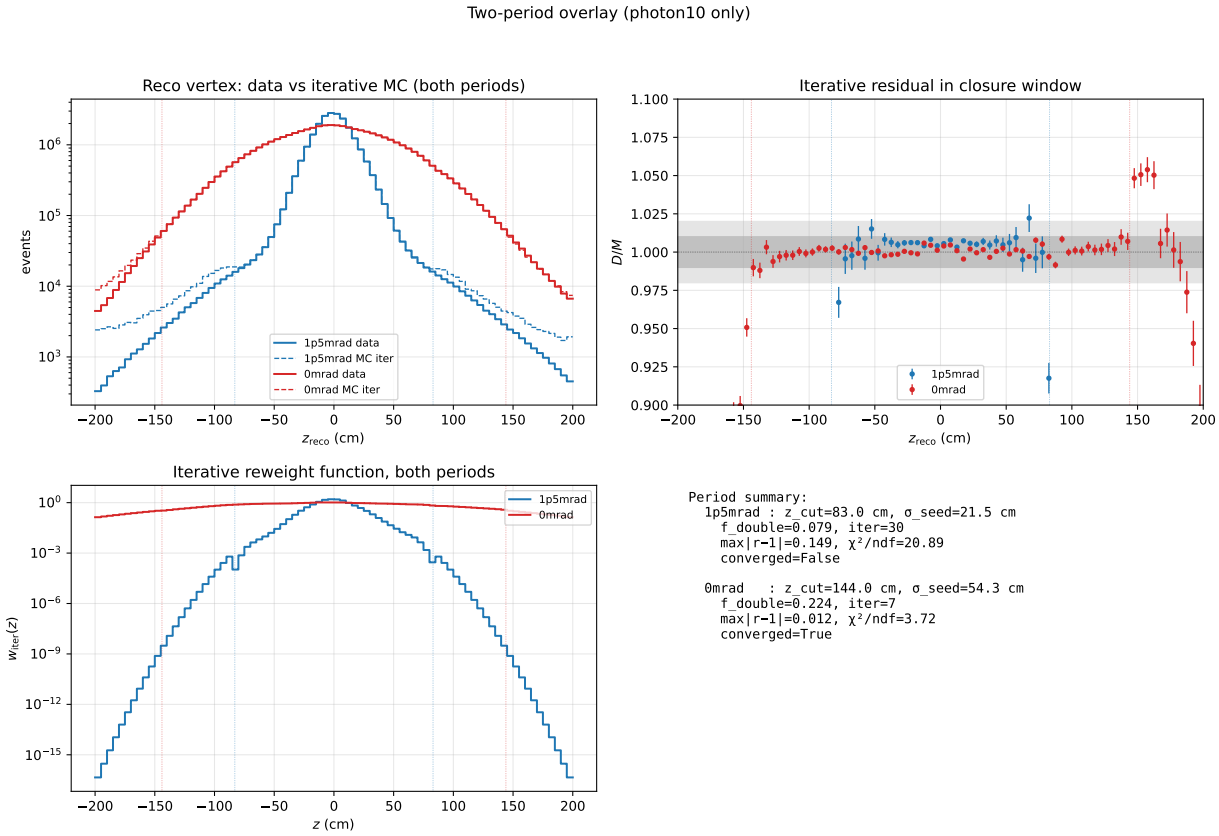

Figure 7: Iterative data-driven reweight summary, both crossing angles. Top row: data reco (black) vs mixed MC reco after iterative reweight (red) for 1.5 mrad and 0 mrad. Middle row: residual $r(z) = D/M_{\text{iter}}$ inside the closure window (vertical dashed lines). Bottom row: recovered $w(z)$, with the seed Gaussian overlaid for context. The 0 mrad case is essentially closed; the 1.5 mrad case is closed in the bulk and stalls at the closure-window edge (see Section 8.4).

0 mrad: clean convergence. Figure 8 shows the 6-panel diagnostic for 0 mrad. The seed Gaussian with $\sigma_{\text{seed}} \simeq 54$ cm already captures most of the shape (initial $\max|r-1| = 8\,\%$, initial $\chi^2/\text{ndf} \simeq 467$); the iteration cleans up the residual over 7 updates and drops below the 1 % tolerance at iteration 6. The recovered $w(z)$ is smooth and broadly Gaussian, with a small enhancement of the central core relative to the seed — this is consistent with the data luminous region being slightly more peaked than a single Gaussian. A stat-only pull diagnostic is written to the fit output directory (`output/0mrad/pull_distribution.pdf`).

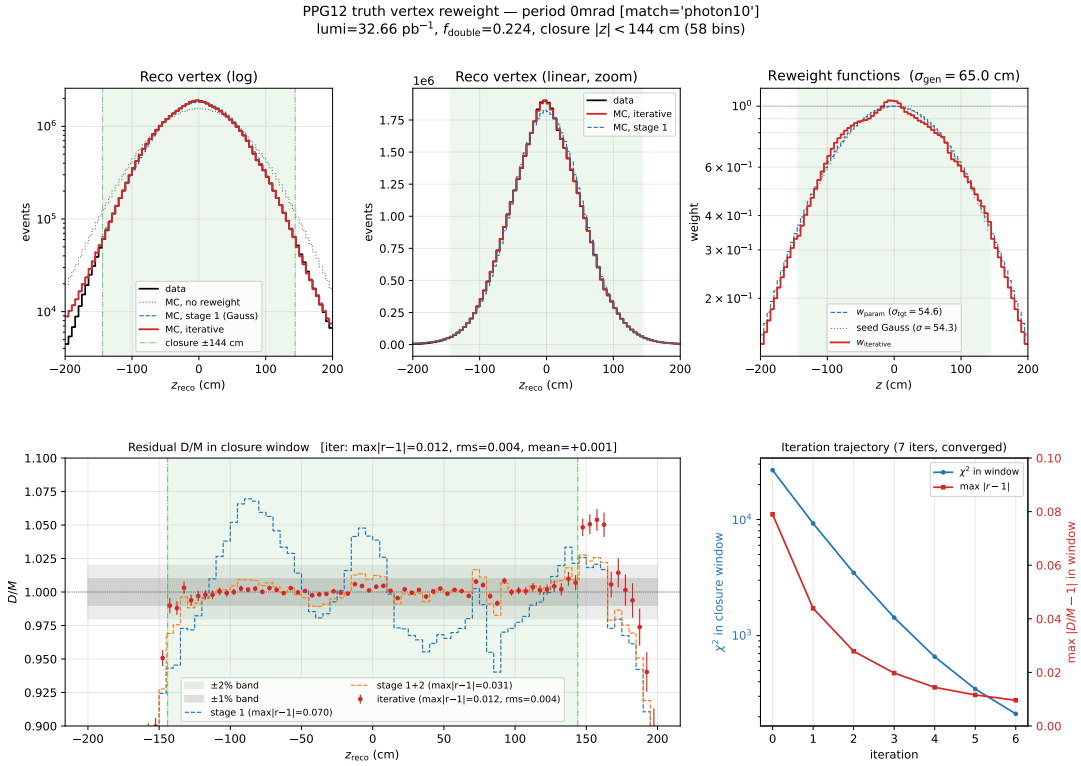

Figure 8: 0 mrad iterative closure (6 panels). Top-left: data reco vs mixed MC reco after iteration. Top-middle: residual inside the closure window. Top-right: recovered $w(z)$ vs seed Gaussian. Bottom-left: per-iteration max $|r-1|$ trajectory. Bottom-middle: per-iteration χ^2. Bottom-right: comparison of the iterative reweight to no-reweight, parametric Gaussian (stage 1), and stage 1+2. The iterative method reaches the 1 % per-bin tolerance at iteration 6, after a total of 7 iterations (the tolerance is checked at the end of each update).

1.5 mrad: bulk closure, edge stall. Figure 9 shows the 6-panel diagnostic for 1.5 mrad. The story is qualitatively different. The seed Gaussian with $\sigma_{\rm seed}\simeq 21$ cm starts from max $|r-1|=63$ % and an in-window χ^2 of 1.4×10^6; the first $\sim$5 iterations make dramatic progress, then the iteration enters a slow plateau driven by a small cluster of bins at the closure-window edge $|z|\simeq 83$ cm (one dominant, one or two others close behind). After 30 iterations, χ^2 in the window has dropped by 2060× to 689 (in-window $\chi^2/{\rm ndf}=20.9$, ndf 33), but max $|r-1|$ sits at 0.15, set by those stuck edge bins. Inside the bulk $|z|<60$ cm closure is at the $\sim$1–2 % level on every bin — see the residual panel of Figure 9. The recovered $w(z)$ shows a strong central core boost ($w(0)\simeq 8$), a mid-z dip, and a clear ± asymmetry in the $|z|\simeq$40–60 cm region that no symmetric Gaussian ansatz can represent.

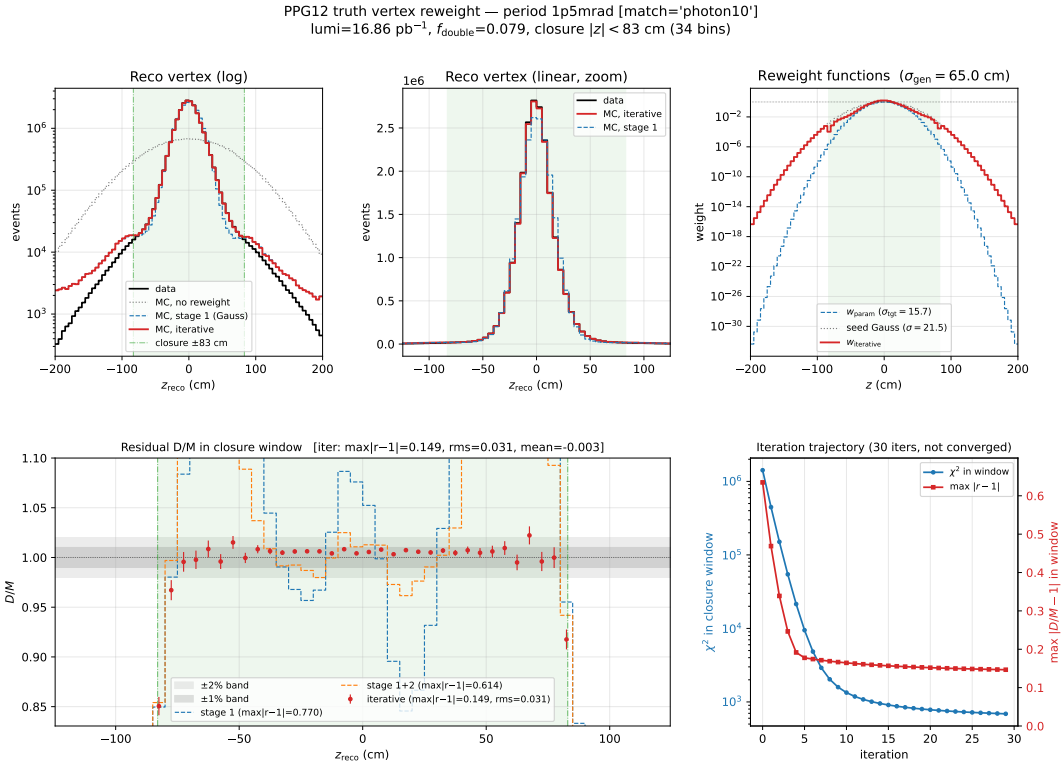

Figure 9: 1.5 mrad iterative closure (6 panels). Same layout as Figure 8. The bulk of the residual closes to $\sim$1–2 % inside $|z| < 60$ cm; the iteration stalls on a small cluster of bins at the closure-window edge $|z| \simeq 83$ cm where the double-interaction MBD resolution kernel starts to dominate the bin content (Section 8.4). The recovered $w(z)$ in the top-right panel shows a central core boost, a mid-z dip, and a clear $\pm$ asymmetry in the $|z| \simeq 40$–60 cm region that no symmetric Gaussian ansatz can represent.

Figure 10 shows the convergence trajectory in detail: $\max|r-1|$ and χ^2 as a function of iteration, on a log-log scale. The two curves track the same physical story — a fast power-law drop for the first $\sim$5 updates, then a near-flat plateau where each additional iteration moves χ^2 by $\lesssim 1$ %.

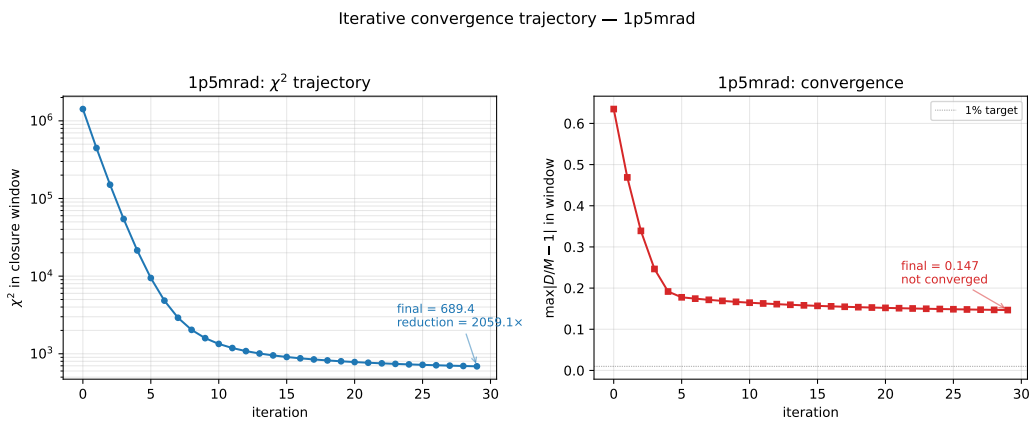

Figure 10: 1.5 mrad convergence trajectory: $\max|r-1|$ in the closure window (top) and in-window χ^2 (bottom) as a function of iteration. The iteration makes dramatic progress in the first 4–5 updates, then enters a plateau where a small cluster of stuck edge bins near $|z| \simeq 83$ cm prevents further $\max|r-1|$ progress. The 30-iteration cap is hit without reaching the 1 % convergence tolerance.

Method comparison. Figure 11 shows the four reweight methods ranked on the 0 mrad mix: no reweight, parametric Gaussian (*stage 1*, equivalent to the closed-form 2D Gaussian of Section 6 restricted to a single-parameter σ fit on the truth axis), parametric plus residual correction (*stage 2*), and the iterative method. The ordering is

$$\text{iterative} \;<\; \text{stage 1+2} \;<\; \text{stage 1} \;<\; \text{no reweight},$$

with each step bringing measurable improvement. The analogous plot at 1.5 mrad is Figure 12: the same ordering holds but with much larger gaps between the methods — the parametric stage 1 alone gives $\chi^2/\mathrm{ndf} \simeq 3237$ in the closure window, and the iterative reweight brings it down to 20.9, a $\sim 155\times$ improvement on top of the parametric fit.

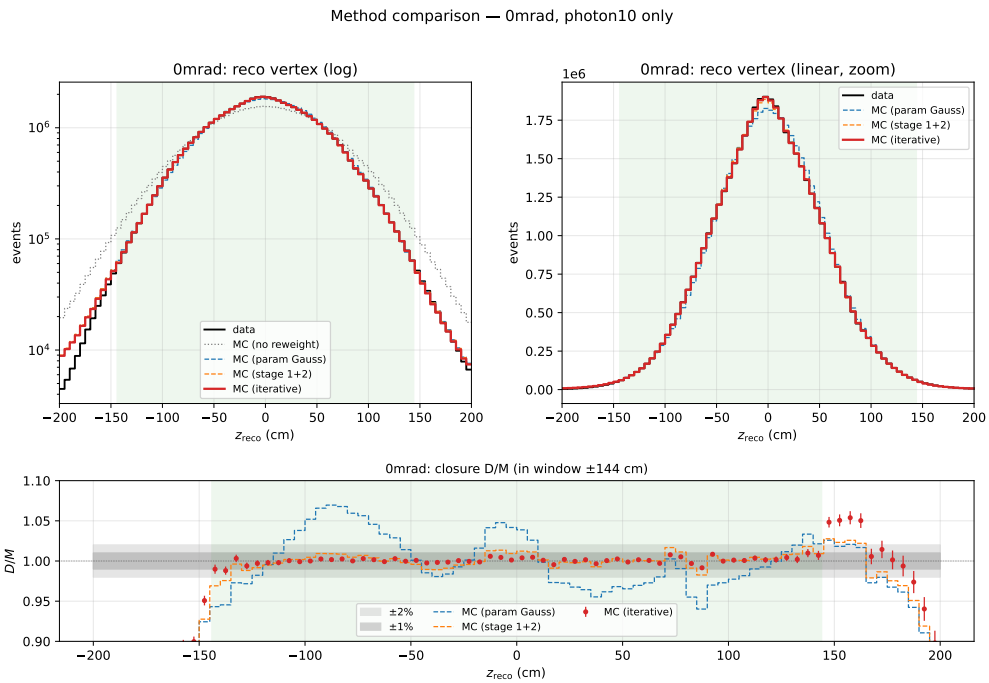

Figure 11: Method comparison at 0 mrad: data reco vs mixed MC reco under no reweight (gray), parametric Gaussian stage 1 (blue), parametric plus residual stage 2 (green), and the iterative data-driven reweight (red). Ratio panel below. The iterative method is the closest match across the closure window; all four agree at the 1–2 % level inside the bulk, with the parametric stages diverging mildly at $|z| \gtrsim 100$ cm.

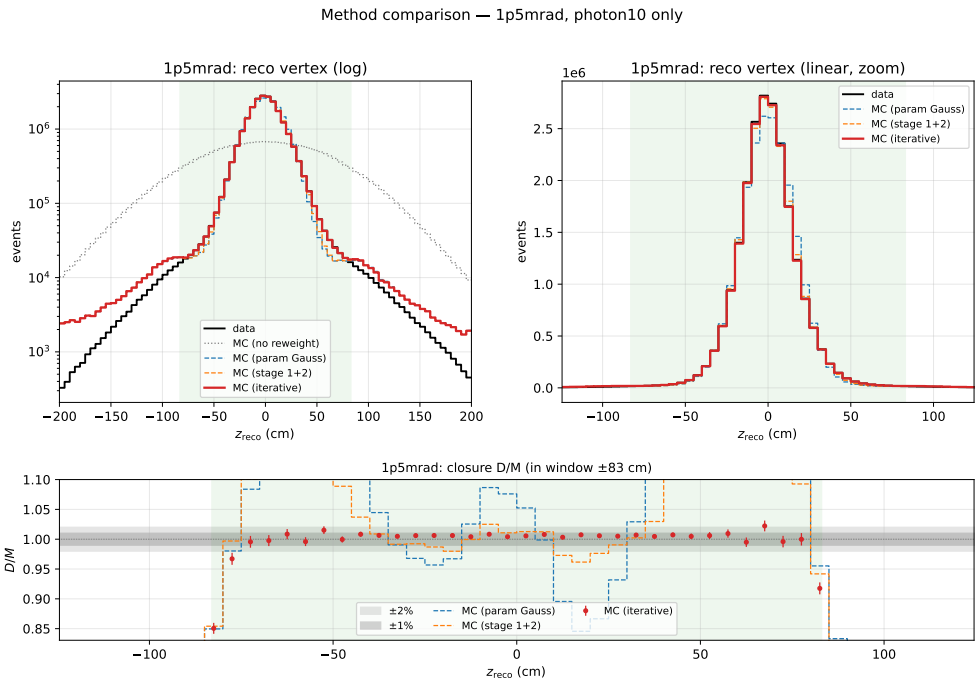

Figure 12: Method comparison at 1.5 mrad, same layout as Figure 11. The spread between methods is much larger here. Stage 1 (blue) has a visible +50 cm bump in the ratio panel and a global $\chi^2/\mathrm{ndf} \simeq 3237$; stage 1+2 (green) reduces the bump partially but still leaves $\sim 60\,\%$ residuals at the edges; the iterative method (red) closes the bulk $|z| < 60$ cm to 1–2 % and stalls only at the edge. The headline number is the $\sim 155\times$ in-window χ^2/ndf improvement from stage 1 to iterative.

8.4 Physics interpretation

Why 0 mrad is easy. At 0 mrad the beams overlap head-on along their full length and the luminous region is broad ($\sigma_{\mathrm{lum}} \simeq 54$ cm), nearly equal to the generator beam profile ($\sigma_{\mathrm{gen}} \simeq 65$ cm). The shape of the data reco vertex is also close to a single Gaussian. The seed Gaussian is therefore already a good first approximation, the initial residual is $\sim 8\,\%$, and the iteration just cleans up small departures (slightly enhanced central core). Convergence in a handful of iterations is what one would expect from a near-textbook deconvolution problem.

Why 1.5 mrad is hard. At 1.5 mrad three things conspire:

- The Piwinski crossing-angle effect compresses the luminous region to $\sigma_{\mathrm{lum}} \simeq 21$ cm, almost a factor of three narrower than the generator profile. The reweight has to extract a narrow target from a wide source, and the dynamic range of $w(z)$ at the central core is correspondingly large ($w(0) \simeq 8$).
- The data reco distribution is *not* a clean Gaussian. It has a narrow core plus a $\pm$-asymmetric mid-tail bump near $|z_r| \simeq 45$–60 cm. The iterative method discovers this asymmetry directly — the recovered $w(z)$ is asymmetric in the same z range — whereas a single-parameter Gaussian (the existing report's stage 1, equivalent to the closed-form 2D Gaussian fit of Section 6) cannot represent it. This is exactly why stage 1 alone gives $\chi^2/\mathrm{ndf} \simeq 3237$ on the 1.5 mrad mix, and why the iterative method drops it to $\simeq 21$.
- The MBD resolution tail leak from core-truth events fills the bins near the closure-window edge ($|z| \simeq 83$ cm), and the iteration becomes insensitive to $w(z)$ updates at that z: contributions to those bins come dominantly from *other* truth-z values via the wide double-interaction

reco-truth kernel. The iteration has the wrong handle on those bins, and that is what sets the residual floor at $\max|r-1| \simeq 15\,\%$.

What the recovered $w(z)$ looks like. At 1.5 mrad, $w(z)$ has three distinguishing features visible in the top-right panel of Figure 9: (a) a strong central peak with $w(0) \simeq 8$ that pulls double-interaction events whose truth vertices lie close to the IP into the data-like core; (b) a mid-z dip that suppresses the generator's natural $|z| \sim 30$–50 cm population to match the narrow data core; and (c) a clear $\pm$ asymmetry in $|z| \simeq 40$–60 cm tracking the asymmetric mid-tail bump in the data. None of these features is representable by a symmetric single-parameter Gaussian. The 2D Gaussian proposal of Section 6 captures (a) and (b) implicitly through the closed-form ratio of Gaussians, but cannot represent (c). At 0 mrad, $w(z)$ is essentially the seed Gaussian with a small central enhancement, and the symmetric Gaussian ansatz is a good approximation.

Relationship to the 2D Gaussian proposal. The iterative method is a direct generalization of the 2D Gaussian proposal (Section 6). Both use the factorization $w(z_{t,1})\,w(z_{t,2})$ for double-interaction events that comes from the algebraic argument of Section 4, both operate on truth vertices, and both close all three vertex axes (primary truth, secondary truth, reco) by construction. The difference is that the 2D Gaussian assumes a closed-form Gaussian ratio with $(\sigma_{\mathrm{MC}}, \sigma_{\mathrm{lum}})$ taken as scalar inputs, while the iterative method discovers $w(z)$ on the bin grid from the data. When the data luminous region is well approximated by a single Gaussian, the iterative $w(z)$ *reduces* to the 2D Gaussian form (the seed already does most of the work; iterations are small corrections, as seen at 0 mrad). When the data is not Gaussian, the iterative method generalizes naturally and the closed-form proposal does not.

Why parametric stage 1 fails at 1.5 mrad. The single-parameter Gaussian ansatz has only σ_{tgt} to play with. It can match the central core (by setting $\sigma_{\mathrm{tgt}} \sim 16$ cm, which is the data-fitted value that minimizes the global χ^2) or the tails (by setting it larger), but not both. With the data showing a narrow core + a non-Gaussian mid-tail bump, the best-fit single Gaussian is incompatible with the bin-by-bin shape at the few-tens-of-percent level: $\chi^2/\mathrm{ndf} \simeq 3237$ in the closure window. Adding the parametric residual correction (stage 2) reduces this substantially but still leaves $\sim 60\,\%$ residuals. The iterative method, with 80 free parameters (one w per bin, smoothed and damped to suppress overfitting), finally captures the actual data shape.

8.5 Caveats and next steps

(i) photon10-only mix. The current fit uses photon10 + photon10_double only. The full production blend also includes jet12 + jet12_double (jet sample weights, finer truth z shape on the jet side). Extending the fit to all four samples is the obvious next step; mechanically it is two more `load_sample` calls plus the appropriate cluster-weighted blend, but the closure numbers will move and should be remeasured before any production hookup.

(ii) 1.5 mrad did not fully converge. The 30-iteration cap was hit. Two questions are open: (a) whether extending to ~ 100 iterations would push the stuck edge bin to convergence or whether it really is a hard physical floor (the suspicion is the latter, because the stuck bin sits where the double-interaction MBD reco-truth kernel dominates); and (b) whether the failure to converge in $\max|r-1|$ matters for the downstream analysis. The latter question has a fairly clean answer: the PPG12 analysis applies a fiducial cut $|z| < 60$ cm on both the reco and truth vertex (Section 5), and inside that cut the per-bin closure is already at the 1–2 % level. The stuck edge bin at $|z| \simeq 83$ cm is well outside the analysis fiducial. The iteration is therefore good enough for the analysis use case, even though it is not formally converged on the full closure-window metric.

(iii) Downstream hookup is not done here. The output `reweight.root` histograms (`h_w_iterative`, `h_w_seed`, `h_M_iterative`) are written and ready to be consumed by `efficiencytool/RecoEffCalculator_TTre` and `ShowerShapeCheck.C`, but the C++ side has not been touched yet. The hookup is the same shape as the 2D Gaussian proposal hookup of Section 6: read $w(z)$ from the ROOT file and apply it to the per-event MC weight, with double-interaction events getting $w(z_{t,1})\, w(z_{t,2})$. This requires the same `vertexz_truth_second` branch from the staged `CaloAna24.cc` patch to access the secondary truth vertex.

(iv) Implementation can be sped up. The current iteration rebuilds the MC reco histogram from scratch on each pass, which is $\mathcal{O}(N_{\text{events}})$ per iteration. Precomputing z^2 and dropping the per-trial mask rebuild are obvious optimizations and would give a $\sim 10\times$ speedup with no change in result. Not a blocker; just nice-to-have for the all-samples extension.

(v) Relation to the "Final recommendation". The "Final recommendation" of Section 7 proposed the closed-form 2D Gaussian as the method of choice. The iterative data-driven method described here is a direct generalization: at 0 mrad the two methods agree (and the iterative $w(z)$ reduces to the 2D Gaussian form up to small central-core corrections), while at 1.5 mrad the iterative method captures asymmetric structure that the closed-form Gaussian cannot. The pragmatic recommendation is therefore to use the closed-form 2D Gaussian as the default nominal for both periods, and to use the iterative $w(z)$ from `efficiencytool/truth_vertex_reweight/` as the 1.5 mrad systematic variant, taking the difference as a vertex-shape modeling uncertainty. The final number should be quoted after the all-samples mix is run.

A Appendix: supporting material

A.1 Sample-independence cross-check (jet12 vs photon10)

Because the 2D reweight is derived from a property of the beam profile and not of the hard scatter, it must be independent of the physics sample. We tested this by running the full truth-only reweighting pipeline on both `jet12(_double)` and `photon10(_double)` with the same data, preselection, and mix fractions. The weights agree between the two MC samples to $\lesssim 1\,\%$ at the peak, confirming the expected universality.

	$w_1(z{=}0)$		$w_2(z{=}0)$	
variant	jet12	photon10	jet12	photon10
mixed_0mrad	1.203	1.191	1.101	1.121
mixed_1p5mrad	3.493	3.494	3.070	3.011

Table 6: Weight values at $z_{\text{truth}} = 0$ for the two MC samples (truth-only constructions w_1 and w_2).

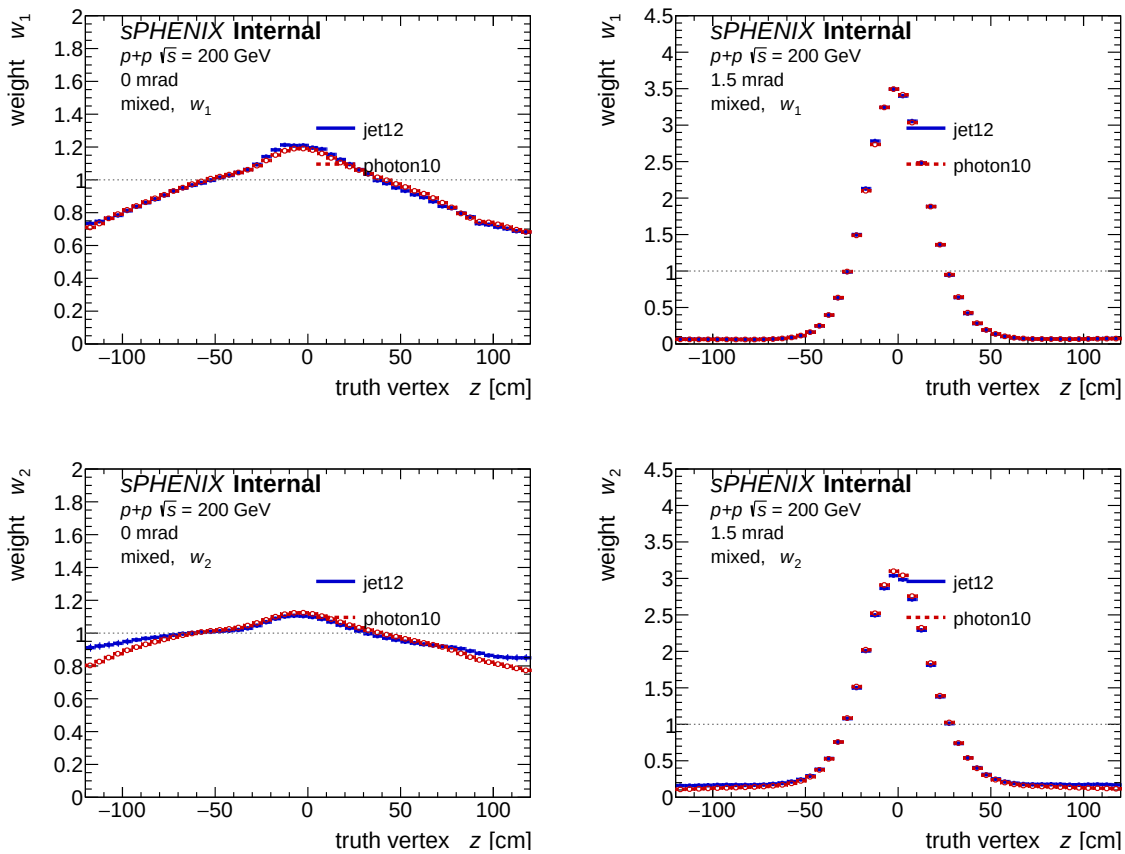

Figure 13: Truth-vertex reweight histograms for jet12 (blue) and photon10 (red dashed) in the mixed variant. Columns: 0 mrad, 1.5 mrad. Rows: w_1 (top), w_2 (bottom). The two samples are indistinguishable at the percent level, confirming that the weight is a beam-profile property.

A.2 Double-interaction width budget and kernel diagnostic

The closure-test failure of the truth-only reweights on double_1p5mrad is entirely geometric and can be read off the numbers:

- 1.5 mrad data reco width: $\sigma \simeq 22.3$ cm
- jet12 single MC truth width: $\sigma \simeq 57.6$ cm
- jet12_double MC truth width (primary): $\sigma \simeq 62.7$ cm
- jet12_double MC reco width: $\sigma \simeq 48.6 \approx 62.7/\sqrt{2}$ (averaging)
- Conditional kernel $P(z_r|z_t{=}0)$ RMS: ~ 34 cm

Figure 14 shows the kernel slice directly, compared to the data target. Even if the truth is forced to a delta at $z_t = 0$, the reco distribution cannot be narrower than the ~ 34 cm kernel width — which is larger than the 22 cm target. Any method that tries to achieve double-MC closure via a truth-only weight is bounded by this.

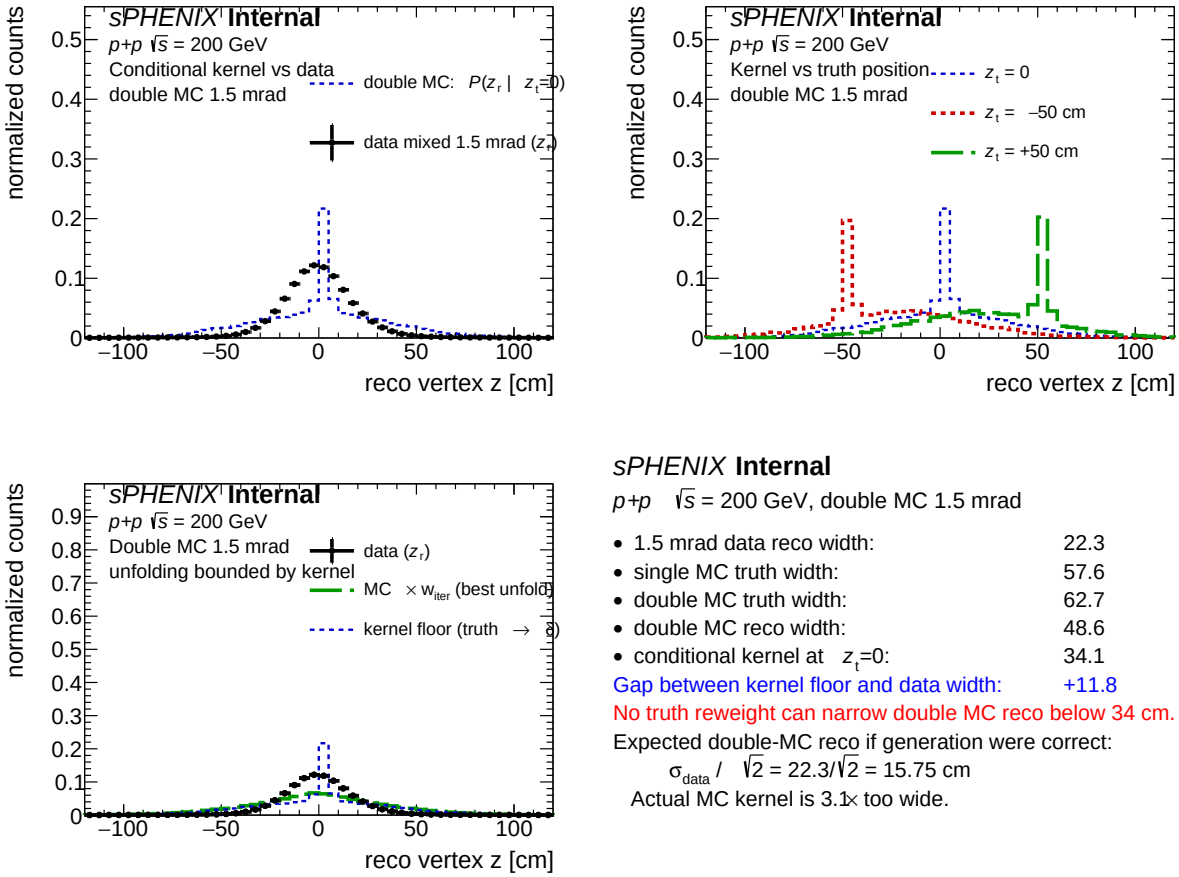

Figure 14: Conditional reco kernel $P(z_r|z_t)$ in jet12_double MC at 1.5 mrad, with the 1.5 mrad data shape overlaid for comparison. The kernel's ~ 34 cm RMS exceeds the ~ 22 cm data peak, demonstrating the geometric lower bound on truth-only reweighting.

A.3 Files and code

Primary macros and outputs:

- `anatreemaker/source/CaloAna24.{h,cc}` — staged patch adding `vertexz_truth_second` (not yet committed).
- `efficiencytool/TruthVertexRecoCheck.C`, `TruthVertexRecoCheckDraw.C` — baseline no-reco-vertex study (Section 2).
- `efficiencytool/VertexReweightAlt.C` (jet12) and `VertexReweightAltPhoton.C` (photon10) — truth-only reweight builders and D'Agostini unfolding (Section 3).
- `efficiencytool/VertexReweightAltDraw.C`, `VertexReweightAltCompare.C`, `VertexReweightAltDoubleM` — plotting.
- `efficiencytool/toy_vertex_2d_reweight.py` — numpy toy MC (Section 4).
- Outputs: `results/truth_vertex_reco_check.root`, `results/vertex_reweight_alt_jet.root`, `results/vertex_reweight_alt_photon.root`.